

Univerzita Karlova
Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země



Karolína Drápelová, Ondřej Kusák

Generalizace budov LOD0

Algoritmy počítačové kartografie

Akademický rok 2025/2026

Praha, 2026

1 Zadání

Ze souboru načtete vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LOD0. Testování proveďte nad třemi datovými sadami - historické centrum města, sídliště, izolovaná zástavba.

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle
- PCA

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LOD0 nahraďte obdelníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a porovnejte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budov dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

Krok	Hodnocení
Generalizace budov metodami Minimum Area Enclosing Rectangle a PCA.	15b
<i>Generalizace budov metodou Longest Edge.</i>	<i>+5b</i>
<i>Generalizace budov metodou Wall Average.</i>	<i>+8b</i>
<i>Generalizace budov metodou Weighted Bisector.</i>	<i>+10b</i>
<i>Implementace další metody konstrukce konvexní obálky.</i>	<i>+5b</i>
<i>Ošetření singulárních případů při generování konvexní obálky.</i>	<i>+2b</i>
<i>Načtení vstupních dat ze *.shp.</i>	<i>+10b</i>
Max celkem:	55b

2 Vypracování

2.1 Generalizace budov

Generalizace budov do úrovně detailu LOD0 (Level of Detail 0) patří mezi základní úlohy digitální kartografie a geografických informačních systémů. V praxi se s ní setkáváme při zjednodušování rozsáhlých datových sad (např. ZABAGED) pro přehlednější vizualizaci v menších měřítkách. Cílem je pro daný polygon budovy $B = \{p_1, \dots, p_n\}$ efektivně nalézt reprezentativní obdélník $G(B)$, který zachovává její hlavní směry a celkovou rozlohu.

Zatímco ruční generalizace by byla pro tisíce budov časově neúnosná, automatizované algoritmy využívají geometrické a statistické metody k nalezení dominantního směru polygonu σ . Společným rysem všech implementovaných metod je finální vytvoření opsaného obdélníku a jeho přeškálování tak, aby odpovídal rozloze původní budovy. Škálovací koeficient k je dán poměrem ploch:

$$k = \frac{A_b}{A_m}$$

kde A_b je plocha původní budovy a A_m je plocha neškálovaného obdélníku. Všechny vektory od těžiště k vrcholům obdélníku jsou následně přenásobeny hodnotou $\sqrt{k}$.

2.2 Konstrukce konvexní obálky - Jarvis Scan

Konvexní obálka (Convex Hull) množiny bodů P je nejmenší konvexní polygon, který obsahuje všechny body z P . Slouží jako nezbytný vstup pro další geometrické operace, zejména pro vyhledání minimálního opsaného obdélníku. V této aplikaci byl pro její konstrukci implementován algoritmus Jarvis Scan (známý jako balení dárku).

Algoritmus iterativně hledá body ležící na hranici obálky. Výchozím bodem (pivotem) q je bod s minimální souřadnicí y . Následně se v každém kroku hledá z množiny zbývajících bodů takový bod p_i , který s aktuálním směrem obálky svírá největší vnitřní úhel ω . Algoritmus končí ve chvíli, kdy se jako další bod na obálce vybere opět výchozí pivot q .

Pseudokód Jarvis Scan

Algorithm 1 Jarvis Scan ($P = \{p_1, \dots, p_n\}$)

```
1: Najdi bod  $q \in P$  s minimální souřadnicí  $y$ 
2: Inicializuj obálku  $CH = \{q\}$ 
3:  $p_{curr} = q$ 
4: Definuj pomocný bod  $p_{prev} = (x_q - 1, y_q)$  ▷ Pro počáteční vektor
5: repeat
6:    $\omega_{max} = 0, p_{next} = \text{null}$ 
7:   for každý bod  $p_i \in P, p_i \neq p_{curr}$  do
8:     Spočítej úhel  $\omega$  mezi vektory  $(p_{curr} - p_{prev})$  a  $(p_i - p_{curr})$ 
9:     if  $\omega > \omega_{max}$  then
10:        $\omega_{max} = \omega$ 
11:        $p_{next} = p_i$ 
12:   Přidej  $p_{next}$  do  $CH$ 
13:    $p_{prev} = p_{curr}$ 
14:    $p_{curr} = p_{next}$ 
15: until  $p_{curr} == q$ 
16: return  $CH$ 
```

2.3 Graham Scan

Graham Scan je další z mnoha metod pro vytvoření konvexní obálky. Algoritmus začíná tím, že najde bod, který má nejmenší souřadnici Y . Bude sloužit jako náš počáteční bod, tzv. pivot. Všechny ostatní body se vezmou a seřadí se podle toho, jaký úhel svírají s horizontální osou procházející pivotem. Nyní se vyfiltrují přebytečné body. Pokud více bodů leží na jedné přímce (mají stejný úhel), zajímá nás pouze ten nejvzdálenější, všechny bližší se odstraní. Nyní se vytvoří zásobník a pomocí cyklu `while` se prochází zbytek bodů. Pro každý nový bod se vezmou poslední dva body ze zásobníku a zkontroluje se, zda jsou levotočivé či pravotočivé. Když jsou levotočivé, poslední bod je bodem konvexní obálky. Pokud jsou pravotočivé, poslední bod nemůže být bodem konvexní obálky.

Pseudokód Graham Scan

Algorithm 2 Graham Scan (P)

```
1: if  $|P| < 3$  then
2:   return  $P$  ▷ Ošetření malého počtu bodů
3:  $q \leftarrow$  bod z  $P$  s min.  $Y$ )
4:  $Body \leftarrow P \setminus \{q\}$ 
5: Seřaď  $Body$  vzestupně podle úhlu vůči  $q$ 
6:  $FiltrovaneBody \leftarrow$  prázdný seznam
7: for  $i \leftarrow 0$  to  $|Body| - 1$  do
8:   if  $i < |Body| - 1$  and  $\text{úhel}(Body[i]) = \text{úhel}(Body[i + 1])$  then
9:     continue ▷ Ignoruj bližší kolineární body
10:   $FiltrovaneBody.append(Body[i])$ 
11: if  $|FiltrovaneBody| < 2$  then
12:   return  $q \cup FiltrovaneBody$ 
13:  $S \leftarrow$  prázdný zásobník
14:  $S.push(q)$ 
15:  $S.push(FiltrovaneBody[0])$ 
16: for  $i \leftarrow 1$  to  $|FiltrovaneBody| - 1$  do
17:   $NovyBod \leftarrow FiltrovaneBody[i]$ 
18:  while  $|S| > 1$  and  $\text{Orientace}(S[-2], S[-1], NovyBod) \leq 0$  do
19:     $S.pop()$  ▷ Odstraňuj při pravotočivé zatáčce
20:   $S.push(NovyBod)$ 
21: return  $S$ 
```

2.4 Minimum Area Enclosing Rectangle

Metoda Minimum Area Enclosing Rectangle (MAER/MBR) vyhledává ohraničující obdélník s absolutně nejmenší plochou. Algoritmus pracuje nad konvexní obálkou polygonu. Využívá principu, že minimální opsaný obdélník sdílí alespoň jednu stranu s hranou konvexní obálky.

Pro každou hranu určí její směrový úhel σ a celou konvexní obálku o tento úhel zpětně

orotuje. Následně se pro orotovanou obálku spočítá min-max box (pravoúhlý hraniční obdélník) a jeho plocha. Výsledkem algoritmu je orotovaný obdélník, u kterého byla nalezena nejmenší plocha. Ten je pak v hlavní funkci generalizace přeškálován.

Pseudokód MAER

Algorithm 3 Minimum Area Enclosing Rectangle (B)

```

1:  $CH = \text{JarvisScan}(B)$ 
2:  $A_{min} = \infty$ ,  $MBR = \text{null}$ ,  $\sigma_{min} = 0$ 
3: for každou hranu  $e_i(p_i, p_{i+1}) \in CH$  do
4:    $\sigma_i = \arctan 2(y_{i+1} - y_i, x_{i+1} - x_i)$ 
5:    $CH_{rot} = \text{Rotate}(CH, -\sigma_i)$ 
6:    $MMB = \text{CreateMinMaxBox}(CH_{rot})$ 
7:    $A_i = \text{Area}(MMB)$ 
8:   if  $A_i < A_{min}$  then
9:      $A_{min} = A_i$ 
10:     $\sigma_{min} = \sigma_i$ 
11:     $MBR = MMB$ 
12:  $MBR_{res} = \text{Rotate}(MBR, \sigma_{min})$ 
13: return  $\text{ResizeRectangle}(MBR_{res}, \text{Area}(B))$ 

```

2.5 PCA (Principal Component Analysis)

Analýza hlavních komponent (PCA) je statistická metoda, která hledá směry největšího rozptylu dat. V kartografii se lomové body budovy interpretují jako množina bodů v rovině, z jejichž souřadnic se vytvoří matice a následně kovarianční matice C .

Výpočtem vlastních čísel a vlastních vektorů této matice (např. metodou SVD, Singular Value Decomposition) získáme hlavní osu budovy. Směrový úhel σ je určen vlastním vektorem $\vec{v}$, který náleží největšímu vlastnímu číslu.

Pseudokód PCA

Algorithm 4 PCA ($B = \{p_1, \dots, p_n\}$)

- 1: Extrahuj vektory souřadnic $X = \{x_1, \dots, x_n\}$ a $Y = \{y_1, \dots, y_n\}$
 - 2: Vytvoř matici $A = [X, Y]$
 - 3: Spočítej kovarianční matici $C = \text{cov}(A)$
 - 4: Proveď SVD rozklad matice $C \rightarrow [U, S, V]$
 - 5: Získej dominantní vektor $\vec{v} = V_0$
 - 6: $\sigma = \arctan 2(v_y, v_x)$
 - 7: $B_{rot} = \text{Rotate}(B, -\sigma)$
 - 8: $MMB = \text{CreateMinMaxBox}(B_{rot})$
 - 9: $MBR = \text{Rotate}(MMB, \sigma)$
 - 10: **return** $\text{ResizeRectangle}(MBR, \text{Area}(B))$
-

2.6 Longest Edge

Metoda nejdelší hrany představuje nejjednodušší a výpočetně nejméně náročný přístup k určení dominantního směru. Algoritmus prochází všechny hrany polygonu budovy a pomocí Pythagorovy věty vyhodnocuje jejich euklidovskou délku s_i . Předpokládá se, že nejdelší kompaktní stěna budovy nejlépe definuje její hlavní orientaci σ .

Pseudokód Longest Edge

Algorithm 5 Longest Edge (B)

- 1: $s_{max} = 0, \sigma_{max} = 0$
 - 2: **for** každou hranu $e_i(p_i, p_{i+1}) \in B$ **do**
 - 3: $s_i = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$
 - 4: **if** $s_i > s_{max}$ **then**
 - 5: $s_{max} = s_i$
 - 6: $\sigma_{max} = \arctan 2(y_{i+1} - y_i, x_{i+1} - x_i)$
 - 7: $B_{rot} = \text{Rotate}(B, -\sigma_{max})$
 - 8: $MMB = \text{CreateMinMaxBox}(B_{rot})$
 - 9: $MBR = \text{Rotate}(MMB, \sigma_{max})$
 - 10: **return** $\text{ResizeRectangle}(MBR, \text{Area}(B))$
-

2.7 Wall Average

Wall Average předpokládá, že většina budov má pravoúhlý půdorys. Je to robustní metoda, ale citlivá na „nepravé“ úhly. Algoritmus prochází všechny hrany budovy a zjišťuje, jak moc se odchyľují od pravého úhlu. Prvním krokem je určení směrnic σ_i pro všechny hrany budovy. Následně se pro každý vrchol p_i spočte relativní úhel ω_i mezi hranami:

$$\omega_i = |\sigma_{i,i+1} - \sigma_{i-1,i}|.$$

Následně se vypočte násobek pravého úhlu, tedy kolikrát se do daného úhlu ω_i vleze pravý úhel (k_i). Vypočteme koeficient k_i :

$$k_i = \frac{2\omega_i}{\pi}.$$

Dále je potřeba zjistit, jaká je odchylka hrany od nejbližšího pravého úhlu. Vypočte se tzv. orientovaný zbytek r_i :

$$r_i = (k_i - \text{round}(k_i)) \cdot \frac{\pi}{2}.$$

Konečný hlavní směr budovy σ se vypočítá tak, že vezmeme základní směr a přičteme k němu vážený průměr všech zjištěných odchylek r_i . Váhou v tomto výpočtu je délka dané hrany s_i . Tím se zajistí, že dlouhé stěny ovlivní výsledek mnohem více než krátké výklenky.

$$\sigma = \sigma_1 + \frac{\sum_{i=1}^n r_i \cdot s_i}{\sum_{i=1}^n s_i}$$

Pseudokód Wall Average

Algorithm 6 Wall Average

```

1:  $n \leftarrow |B|$ 
2:  $sum\_rs \leftarrow 0$ 
3:  $sum\_s \leftarrow 0$ 
4:  $\sigma_1 \leftarrow \text{atan2}(p_1.y - p_0.y, p_1.x - p_0.x)$  ▷ Směrnice první hrany
5: for  $i \leftarrow 0$  to  $n - 1$  do
6:    $p_1 \leftarrow B[i]$ 
7:    $p_2 \leftarrow B[(i + 1) \bmod n]$ 
8:    $\Delta x \leftarrow p_2.x - p_1.x$ 
9:    $\Delta y \leftarrow p_2.y - p_1.y$ 
10:   $s_i \leftarrow \sqrt{\Delta x^2 + \Delta y^2}$  ▷ Délka hrany
11:   $\sigma_i \leftarrow \text{atan2}(\Delta y, \Delta x)$  ▷ Směrnice hrany
12:   $\omega_i \leftarrow \sigma_i - \sigma_1$ 
13:   $k_i \leftarrow \frac{2 \cdot \omega_i}{\pi}$ 
14:   $r_i \leftarrow (k_i - \text{round}(k_i)) \cdot \frac{\pi}{2}$ 
15:   $sum\_rs \leftarrow sum\_rs + (r_i \cdot s_i)$  ▷ Suma vážených odchylek
16:   $sum\_s \leftarrow sum\_s + s_i$  ▷ Suma délek stěn
17:  $\sigma_{avg} \leftarrow \sigma_1 + \left( \frac{sum\_rs}{sum\_s} \right)$  ▷ Hlavní směr budovy
18:  $B_{rot} \leftarrow \text{RotatePolygon}(B, -\sigma_{avg})$ 
19:  $(MMB, area) \leftarrow \text{CreateMMB}(B_{rot})$ 
20:  $MBR \leftarrow \text{RotatePolygon}(MMB, \sigma_{avg})$ 
21:  $MBR_{res} \leftarrow \text{ResizeRectangle}(B, MBR)$ 
22: return  $MBR_{res}$ 

```

2.8 Weighted Bisector

Metoda Weighted Bisector je další přístup pro určení hlavního směru budovy. Základní myšlenkou je, že hlavní orientaci a proporce budovy nejlépe vystihují její nejdelší úhlopříčky. Algoritmus nejprve identifikuje všechny možné spojnice (úhlopříčky) mezi vrcholy polygonu. Z nich se následně vyberou dvě absolutně nejdelší úhlopříčky. Pro tyto dvě vybrané nejdelší úhlopříčky se zjistí dva hlavní parametry - jejich délky (s_1 a s_2), jejich směrnice (σ_1 a σ_2). Konečný hlavní směr budovy σ tvoří osu úhlu mezi těmito dvěma úhlopříčkami. Aby delší úhlopříčka měla na výsledek větší vliv, počítá se vážený průměr jejich směrnic, kde váhou je právě délka příslušné úhlopříčky.

$$\sigma = \frac{s_1 \cdot \sigma_1 + s_2 \cdot \sigma_2}{s_1 + s_2}$$

Pseudokód Weighted Bisector

Algorithm 7 Weighted Bisector

```

1:  $n \leftarrow |B|$ 
2:  $Diagonaly \leftarrow$  prázdný seznam
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:   for  $j \leftarrow i + 1$  to  $n - 1$  do
5:      $p_1 \leftarrow B[i]$ 
6:      $p_2 \leftarrow B[j]$ 
7:      $\Delta x \leftarrow p_2.x - p_1.x$ 
8:      $\Delta y \leftarrow p_2.y - p_1.y$ 
9:      $delka \leftarrow \sqrt{\Delta x^2 + \Delta y^2}$ 
10:     $\sigma \leftarrow \text{atan2}(\Delta y, \Delta x)$ 
11:     $\sigma \leftarrow \sigma \bmod \pi$  ▷ Normalizace, diagonály jsou neorientované
12:     $Diagonaly.append((delka, \sigma))$ 
13: Seřaď  $Diagonaly$  sestupně podle proměnné  $delka$ 
14:  $(s_1, \sigma_1) \leftarrow Diagonaly[0]$  ▷ První nejdelší diagonála
15:  $(s_2, \sigma_2) \leftarrow Diagonaly[1]$ 
16:  $\sigma_{avg} \leftarrow \frac{s_1 \cdot \sigma_1 + s_2 \cdot \sigma_2}{s_1 + s_2}$  ▷ Výpočet váženého průměru úhlů
17:  $B_{rot} \leftarrow \text{RotatePolygon}(B, -\sigma_{avg})$ 
18:  $(MMB, area) \leftarrow \text{CreateMMB}(B_{rot})$ 
19:  $MBR \leftarrow \text{RotatePolygon}(MMB, \sigma_{avg})$ 
20:  $MBR_{res} \leftarrow \text{ResizeRectangle}(B, MBR)$ 
21: return  $MBR_{res}$ 

```

2.9 Načtení dat ze .shp

Načtení dat se provádí pomocí knihovny *shapefile*. Z důvodů různých souřadnicových systémů (souřadnice obrazovky jsou jiné než souřadnice shapefile) bylo potřeba shapefile přemapovat do souřadnic obrazovky. Z původních shapefile souřadnic byly spočítány minima a maxima x-ových a y-ových souřadnic. Z těchto údajů pak byly spočítány rozměry.

```
1  #Calculate the dimensions
2      shp_width = glob_max_x - glob_min_x
3      shp_height = glob_max_y - glob_min_y
```

Kód 1: Rozměry shapefile.

Ty poté byly vycentrovány na střed obrazovky, zároveň byl zvolen 5 % okraj, aby se budovy nedotýkaly okrajů, bylo zvoleno i měřítko. Následně byly body tvořící původní polygony přemapovány do souřadnic obrazovky a tyto přemapované body vytvořily polygony, které jsou poté načítány v samotné aplikaci.

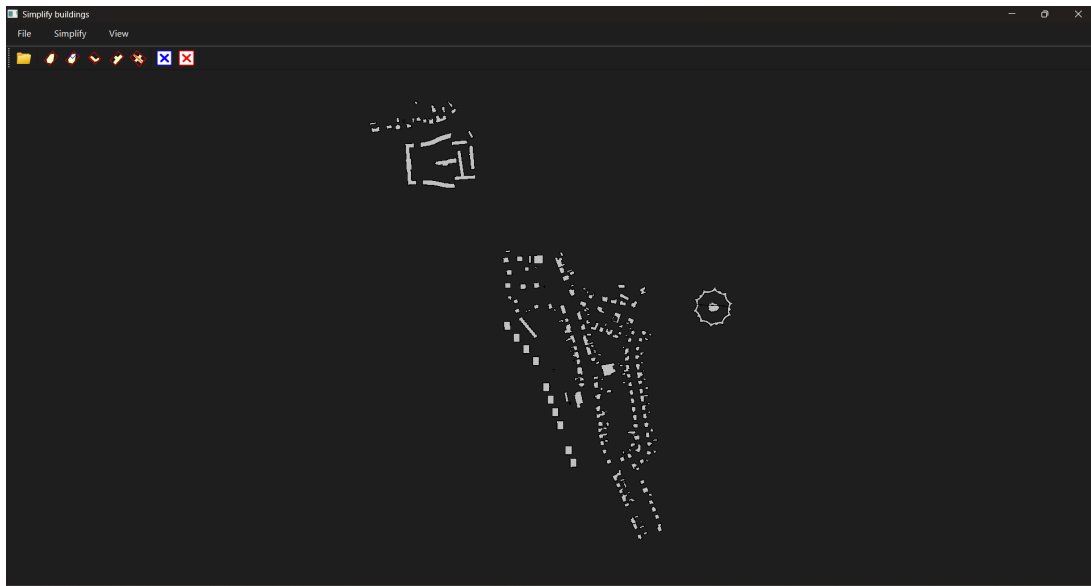
```
1  for points in raw_polygons:
2      scaled_points = []
3      for x, y in points:
4          screen_x = (x - glob_min_x) * scale + x_offset
5          screen_y = canvas_height - ((y - glob_min_y) * scale +
6          y_offset)
7          scaled_points.append(QPointF(screen_x, screen_y))
8      self.__buildings.append(QPolygonF(scaled_points))
```

Kód 2: Přemapování souřadnic bodů/polygonů.

2.10 Vstupní data

Vstupními daty jsou shapefile soubory budov ZABAGED, které byly upraveny v GIS software.

Výstupem je aplikace vytvořená v rozhraní QT, která generalizuje nahraný shapefile budov pomocí metod Minimum Area Enclosing Rectangle, PCA, Longest Edge, Wall Average a Weighted Bisector a zobrazuje, jak generalizace vypadá.



Obrázek 1: Printscreen vytvořené aplikace s načtenými daty.

2.11 Dokumentace

2.11.1 Třída Algorithms

Třída *Algorithms* zahrnuje metody *get2VectorsAngle* - vrátí úhel mezi dvěma vektory, *getOrientation* - vrací orientaci (CW, CCW), *getDistance* - vrací vzdálenost mezi dvěma body, *createCHGraham* - vytvoří konvexní obálku pomocí Graham Scan, *createCH* - vytvoří konvexní obálku pomocí Jarvis Scan, *createMMB* - vytvoří minimum bounding box, *rotatePolygon* - rotuje polygon zadaným úhlem, *createMBR* - vytvoří minimum bounding rectangle, *getArea* - vrátí rozlohu polygonu, *resizeRectangle* - přeškáluje polygon, *simplifyBuildingMBR* - generalizuje polygon pomocí MBR, *simplifyBuildingPCA* - generalizuje polygon pomocí PCA, *simplifyBuildingLongestEdge* - generalizuje polygon pomocí Longest Edge, *simplifyBuildingWallAverage* - generalizuje polygon pomocí Wall Average a *simplifyBuildingWeightedBisector* - generalizuje polygon pomocí Weighted Bisector.

2.11.2 Třída Draw

Třída *Draw* představuje stěžejní vizuální komponentu celé aplikace a slouží jako interaktivní kreslicí plátno (Canvas). Obsahuje metody pro zachytávání uživatelských vstupů, jako je *mousePressEvent*, která umožňuje manuální zadávání lomových bodů polygonu pomocí myši. Samotné vykreslování veškeré grafiky zajišťuje metoda *paintEvent*, která

s využitím nástroje *QPainter* vizuálně odlišuje původní budovy (šedě), konvexní obálky (modře) a výsledné generalizované obdélníky (červeně).

Z hlediska datového toku obsahuje třída sadu přístupových metod. *setMBR*, *setMBRs* a *setCH* slouží pro zápis vypočtených dat (výsledků algoritmů) zpět do plátna, zatímco *getBuilding* a *getBuildings* předávají data o polygonech zpět do výpočetního procesu. Klíčovou součástí třídy je metoda *LoadShapesToScene*, která nejenže čte data z nahraného ESRI Shapefile, ale především zajišťuje nezbytnou transformaci, tedy výpočet měřítko, posun do středu plátna s ochranným okrajem a celkové přemapování globálních souřadnic do lokálního souřadnicového systému obrazovky. Třidu doplňuje metoda *clearResults* pro vyčištění pracovní plochy.

2.11.3 Třída Main Form

Třída *Main Form* plní roli řídicího softwarového uzlu (Controlleru), který spravuje grafické uživatelské rozhraní a spravuje rozložení okna včetně zapouzdřeného kreslicího plátna (*Draw*). Její stěžejní funkcí je propojení vizuální části s výpočetní logikou prostřednictvím obsluhy událostí. Při interakci uživatele třída zprostředkuje tok dat: získá souřadnice budov z plátna, nechá třídu *Algorithms* provést příslušnou generalizaci a vypočtené výsledky odešle zpět k překreslení scény. Třída tak řídí celou aplikaci a striktně odděluje uživatelské ovládání od geometrických výpočtů.

2.12 Výsledky a závěr

V rámci této práce byla úspěšně vytvořena aplikace v prostředí PyQt6, která automatizuje proces generalizace půdorysů budov do úrovně detailu LOD0. Aplikace umožňuje bezproblémové načítání reálných datových sad ve formátu *.shp* (např. ZABAGED) a jejich transformaci do souřadnicového systému obrazovky.

Pro určení dominantního směru budov bylo implementováno pět nezávislých algoritmů: Minimum Area Enclosing Rectangle (MAER), Principal Component Analysis (PCA), Longest Edge, Wall Average a Weighted Bisector. Součástí řešení je i robustní výpočet konvexní obálky s využitím algoritmů Jarvis Scan a časově efektivnějšího Graham Scan. Významným prvkem programu je ošetření singulárních případů (kolineární body, polygony s nulovou plochou), díky kterému aplikace nepadá ani na topologicky nekorektních datech. Všechny metody pak tedy zachovávají původní rozlohu budovy díky aplikovanému plošnému škálovacímu koeficientu k .

Porovnání metod a diskuze výsledků

Při testování na reálných datech (zástavba historického centra, moderní sídliště, izolované rodinné domy) se ukázalo, že neexistuje jeden univerzálně nejlepší algoritmus. Každá metoda se hodí pro jiný typ zástavby i s ohledem na její rozložení:

- **MAER / MBR** poskytuje vizuálně velmi stabilní obdélníky. Z principu své konstrukce však může u budov ve tvaru písmene "L", "U", nebo u diagonálně orientovaných bloků generovat neúměrně široké a zkreslené výsledky, které vizuálně nerespektují hmotu budovy.
- **PCA** je výpočetně velmi rychlá metoda, která se hodí především na protáhlé liniové budovy a sídlištní zástavbu. Ukázala se však jako náchylná na drobné a asymetrické výstupky z jinak pravoúhlého půdorysu, které dokážou hlavní osu nechtěně vychýlit.
- **Longest Edge** funguje spolehlivě u jednoduché izolované zástavby bez složitých detailů. Výrazně ale selhává u složitějších tvarů (např. historická zástavba), kde náhodná nejdelší kompaktní stěna nemusí vůbec reprezentovat reálnou orientaci celé budovy.
- **Wall Average** se ukázal jako mimořádně robustní algoritmus pro budovy tvořené

velkým množstvím malých, na sebe kolmých stěn. Tím, že průměruje směry s využitím vah (délek), eliminuje chyby, které u takových budov dělá metoda Longest Edge.

- **Weighted Bisector** poskytuje výborné výsledky při aproximaci složitých tvarů, kde selhávají metody založené výhradně na analýze vnějších obvodových stěn. Protože analyzuje vnitřní strukturu budovy (úhlopříčky), dokáže dobře zachytit její celkový tvarový a težištní trend.

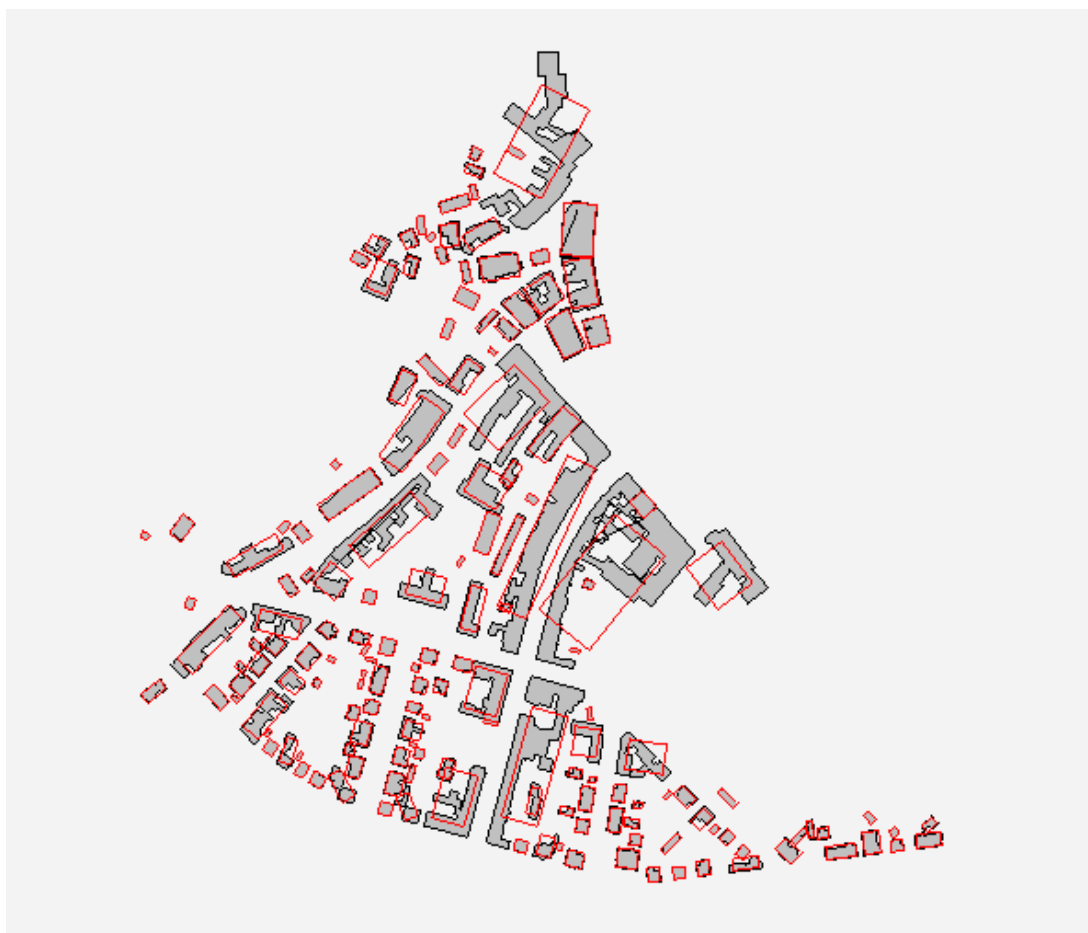
Na závěr můžeme uvést, že aplikace splnila všechny zadané požadavky a představuje plně funkční a rozšiřitelný nástroj pro automatizovanou plošnou generalizaci. **Grafická ukázka výsledků jednotlivých metod je vyobrazena v příloze tohoto dokumentu**

3 Literatura, zdroje a přílohy

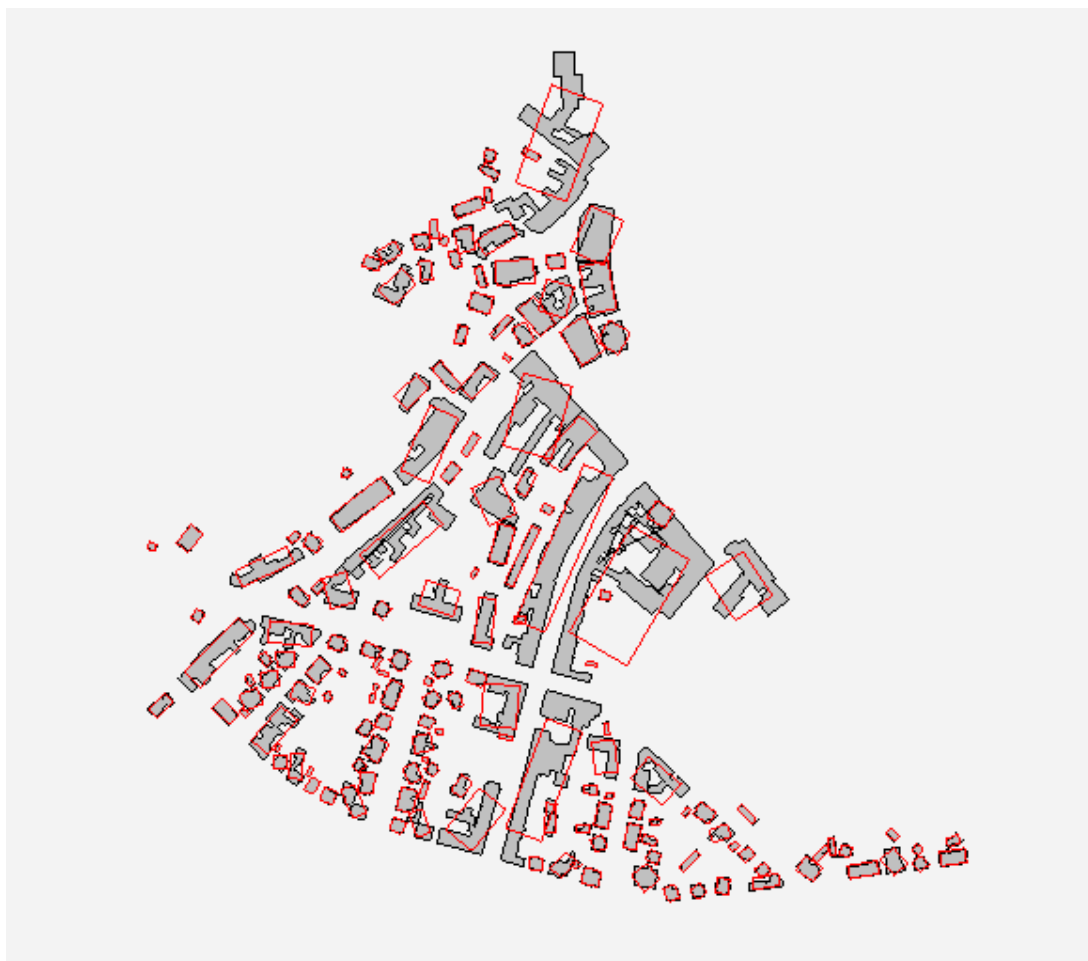
BAYER, T. (2008): Algoritmy v digitální kartografii. Nakladatelství Karolinum, Praha.

BAYER, T. (2026): Konvexní obálka množiny bodů. Přednáška. Katedra aplikované geoinformatiky a kartografie. Přírodovědecká fakulta UK (cit. 3. 4. 2026). Dostupné z: https://web.natur.cuni.cz/~bayertom/images/courses/Adk/adk4_new.pdf.

Přílohy s ukázkami výsledků jednotlivých metod



Obrázek 2: Printscreen výsledků generalizace metodou MBR nad daty ze starého města.



Obrázek 3: Printscreen výsledků generalizace metodou PCA nad daty ze starého města.



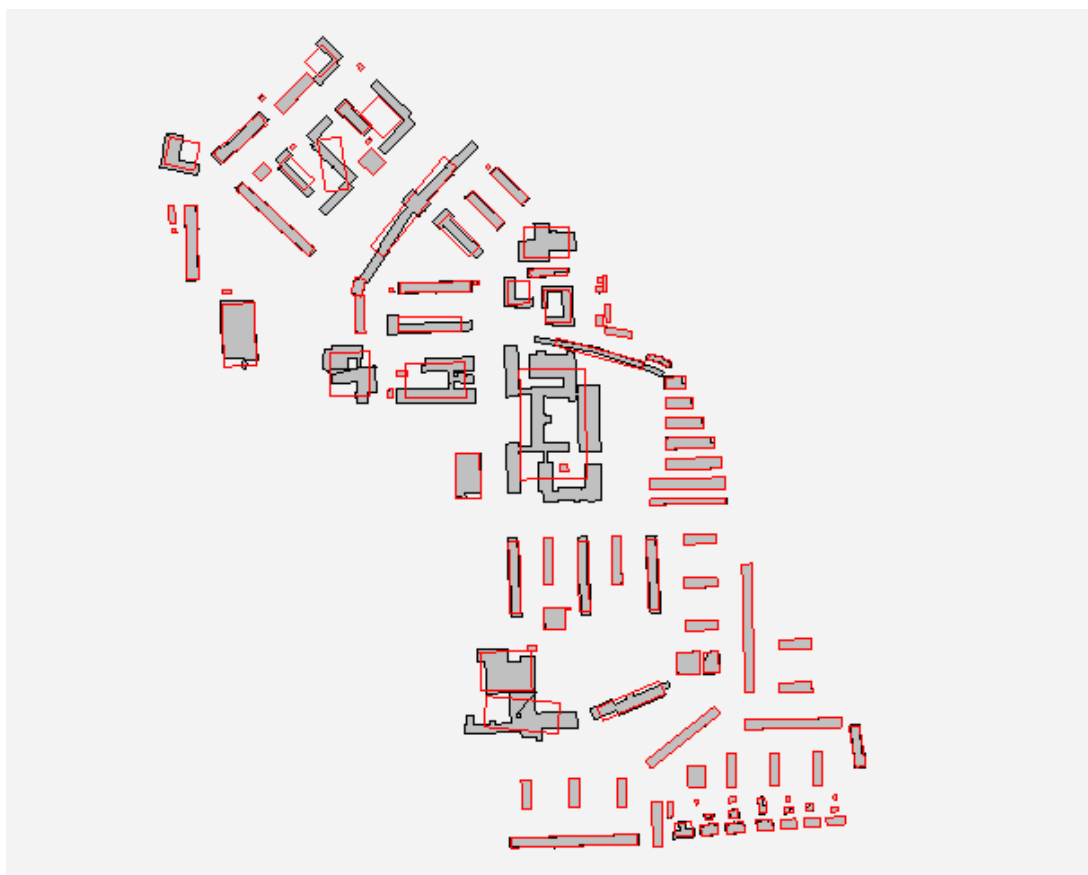
Obrázek 4: Printscreen výsledků generalizace metodou Longest edge nad daty ze starého města.



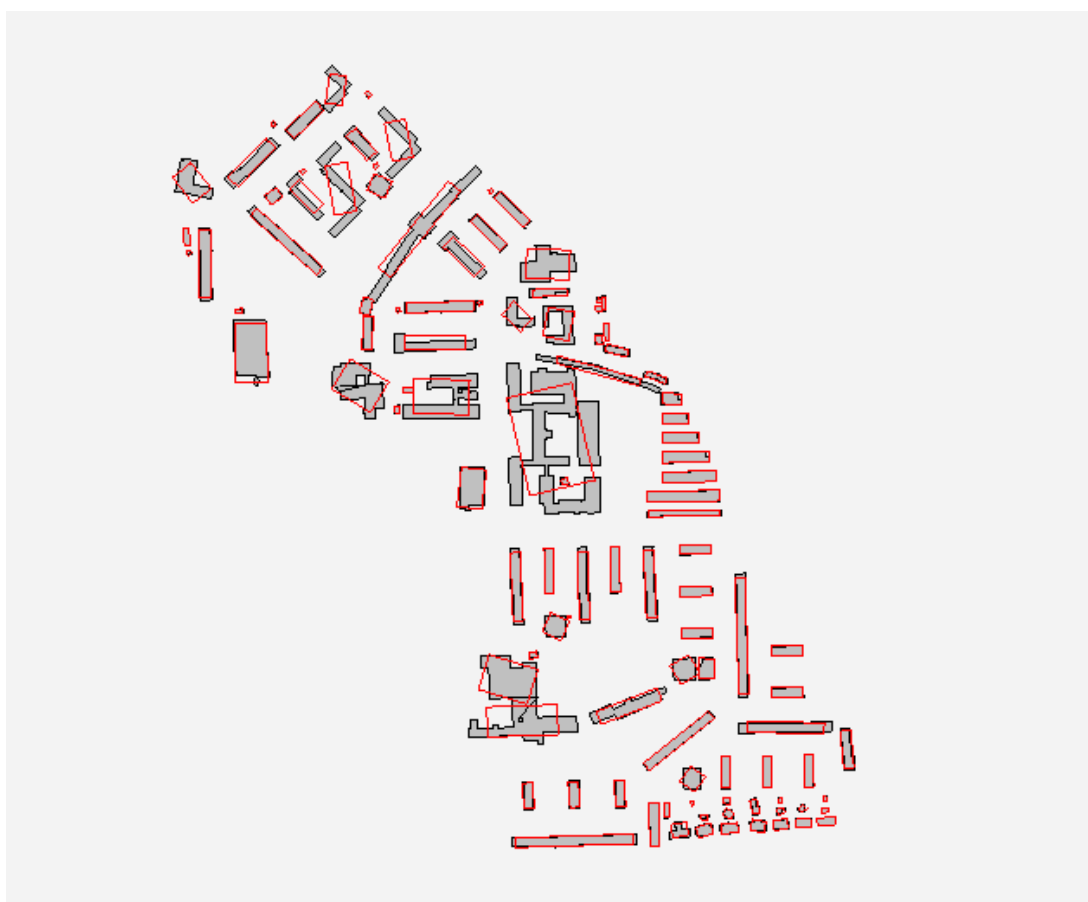
Obrázek 5: Printscreens výsledků generalizace metodou Wall Average nad daty ze starého města.



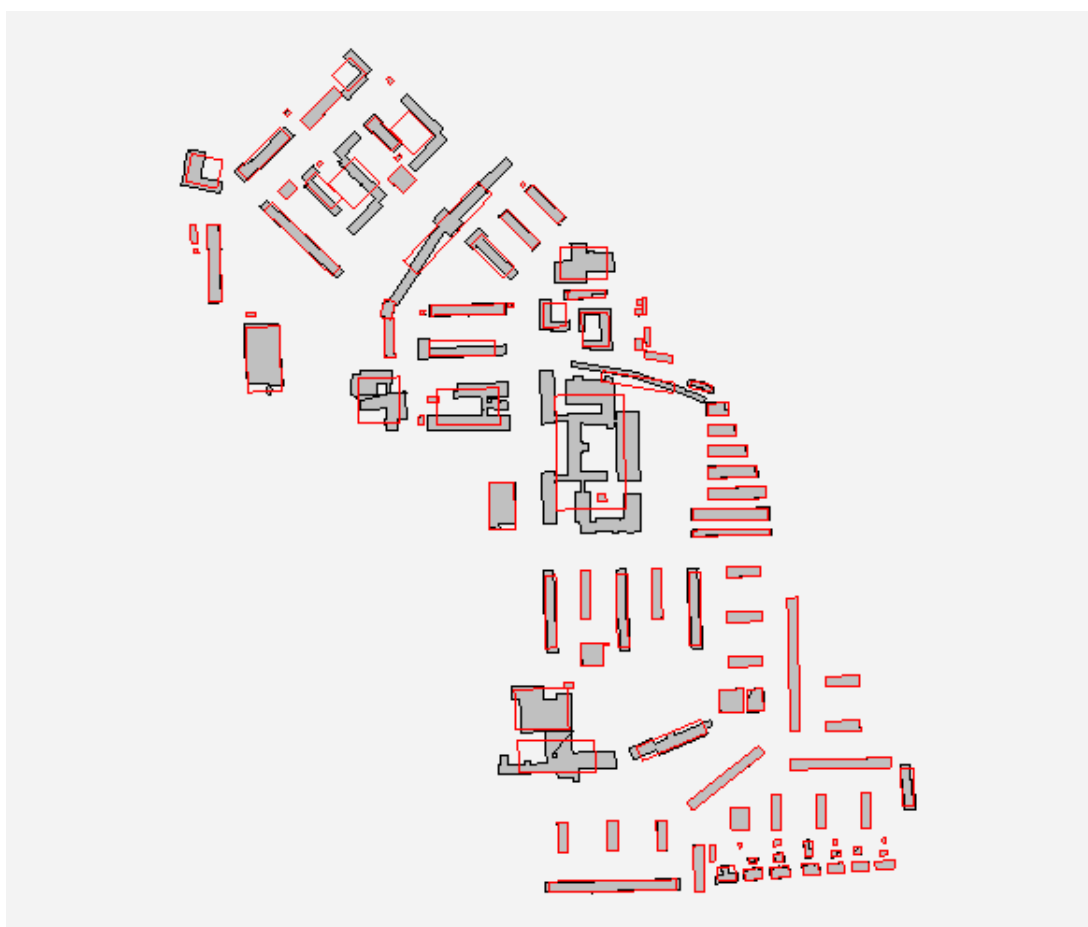
Obrázek 6: Printscreen výsledků generalizace metodou Weighted Bisector nad daty ze starého města.



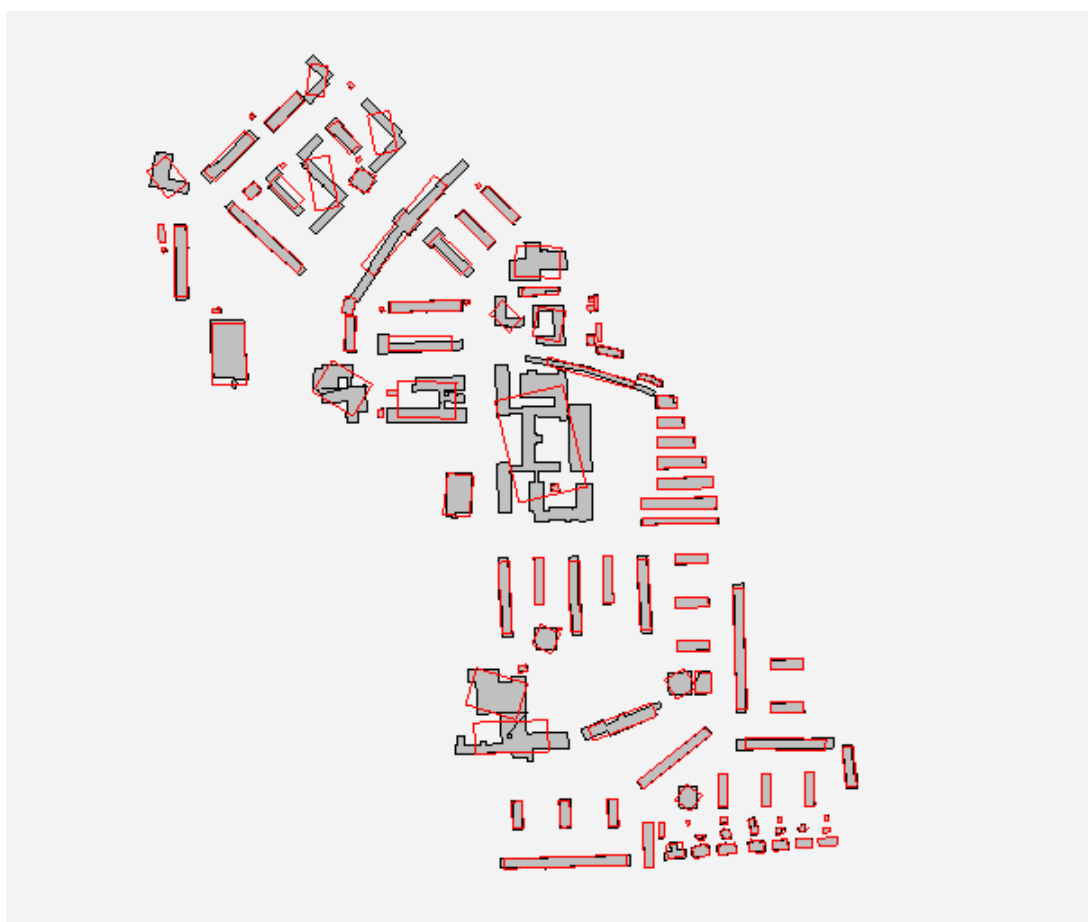
Obrázek 7: Printscreen výsledků generalizace metodou MBR nad daty ze sídliště.



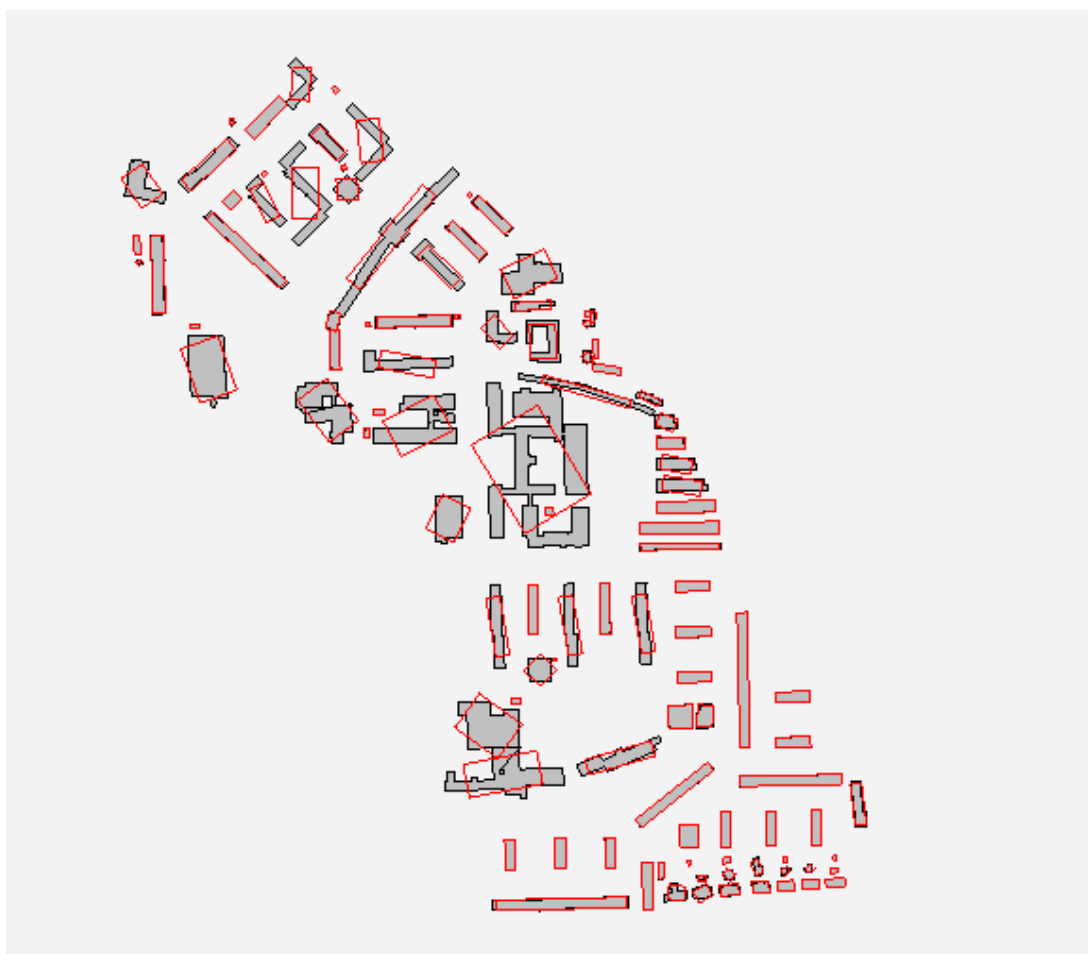
Obrázek 8: Printscreen výsledků generalizace metodou PCA nad daty ze sídliště.



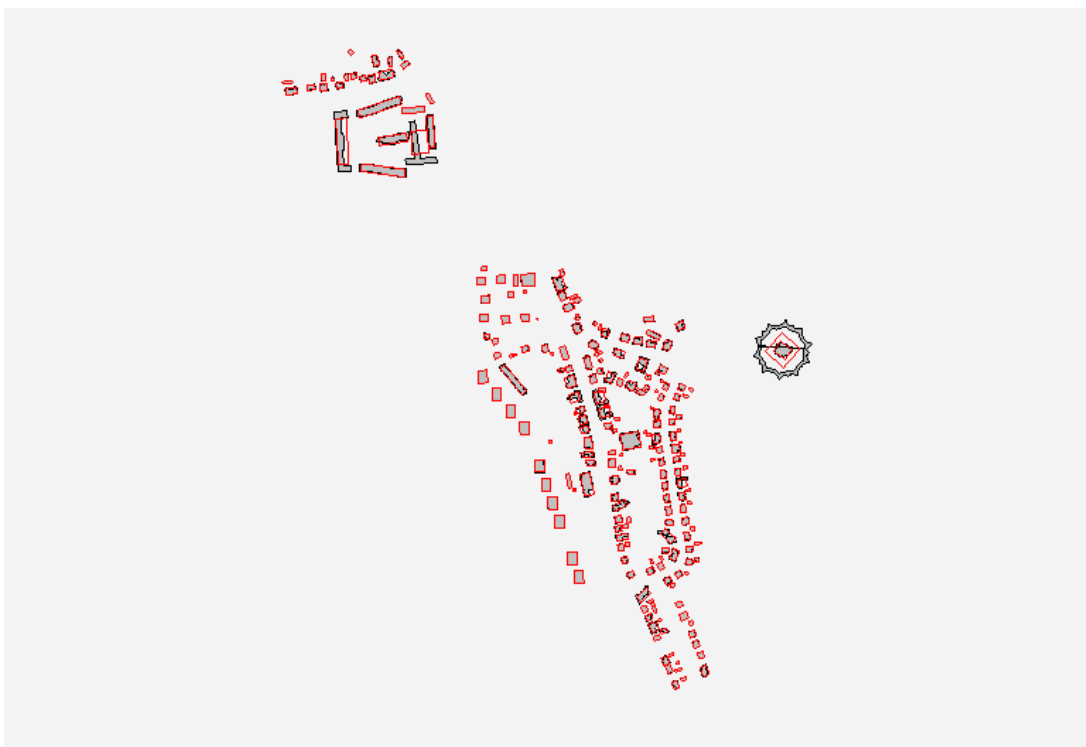
Obrázek 9: Printscreen výsledků generalizace metodou Longest edge nad daty ze sídliště.



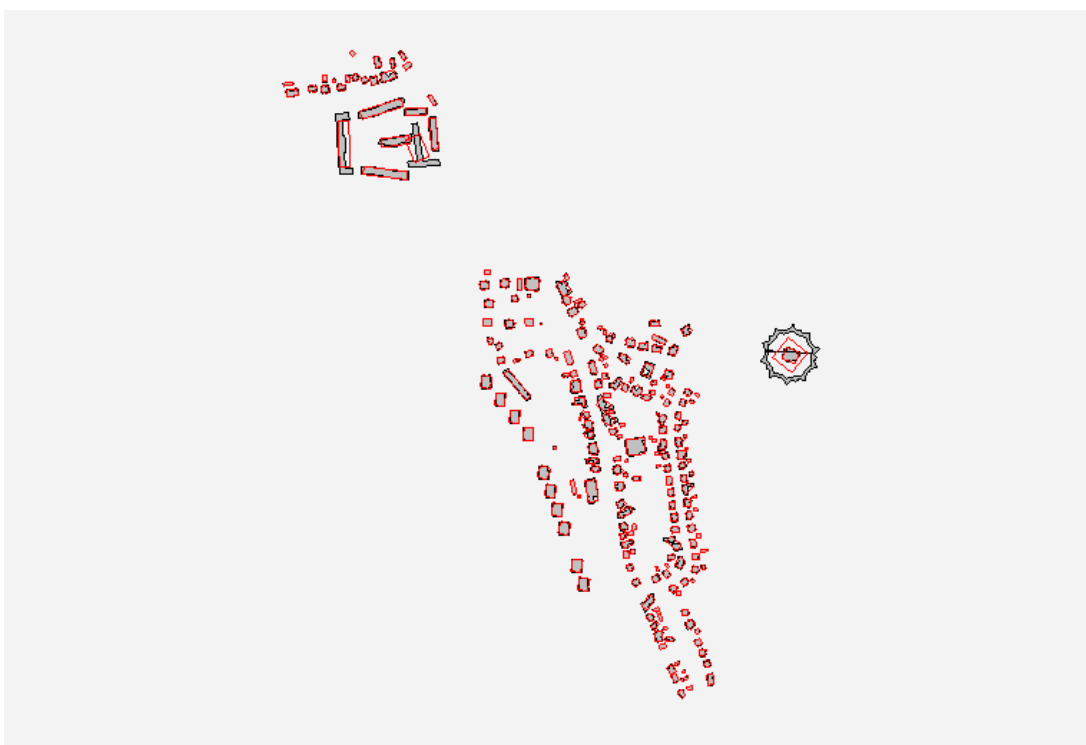
Obrázek 10: Printscreen výsledků generalizace metodou Wall Average nad daty ze sídliště.



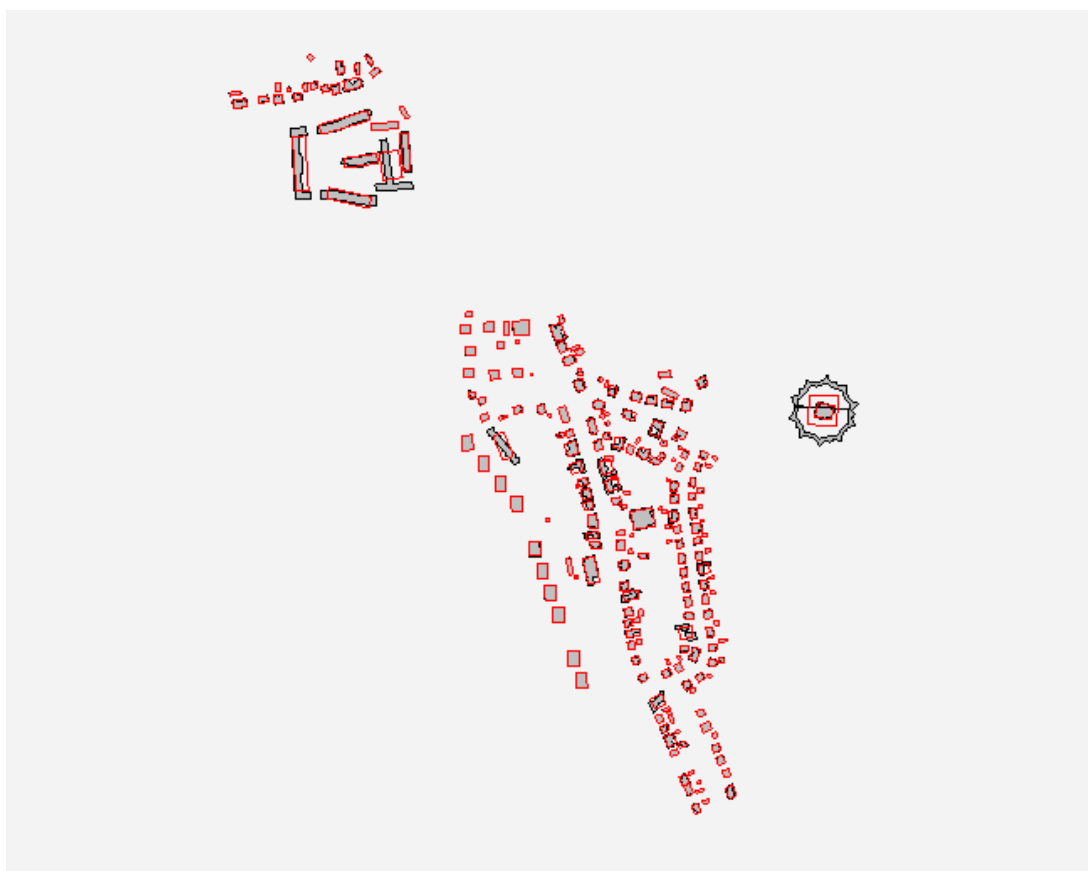
Obrázek 11: Printscreen výsledků generalizace metodou Weighted Bisector nad daty ze sídliště.



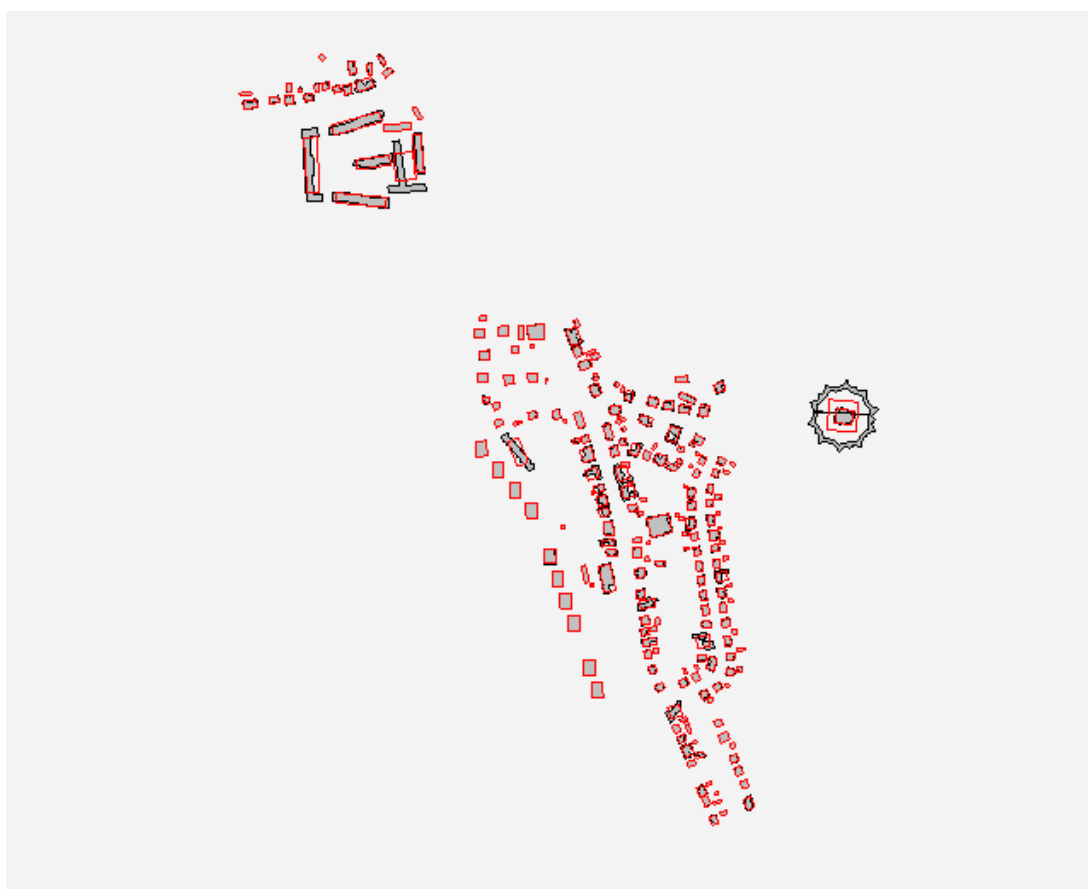
Obrázek 12: Printscreen výsledků generalizace metodou MBR nad daty z izolované zástavby.



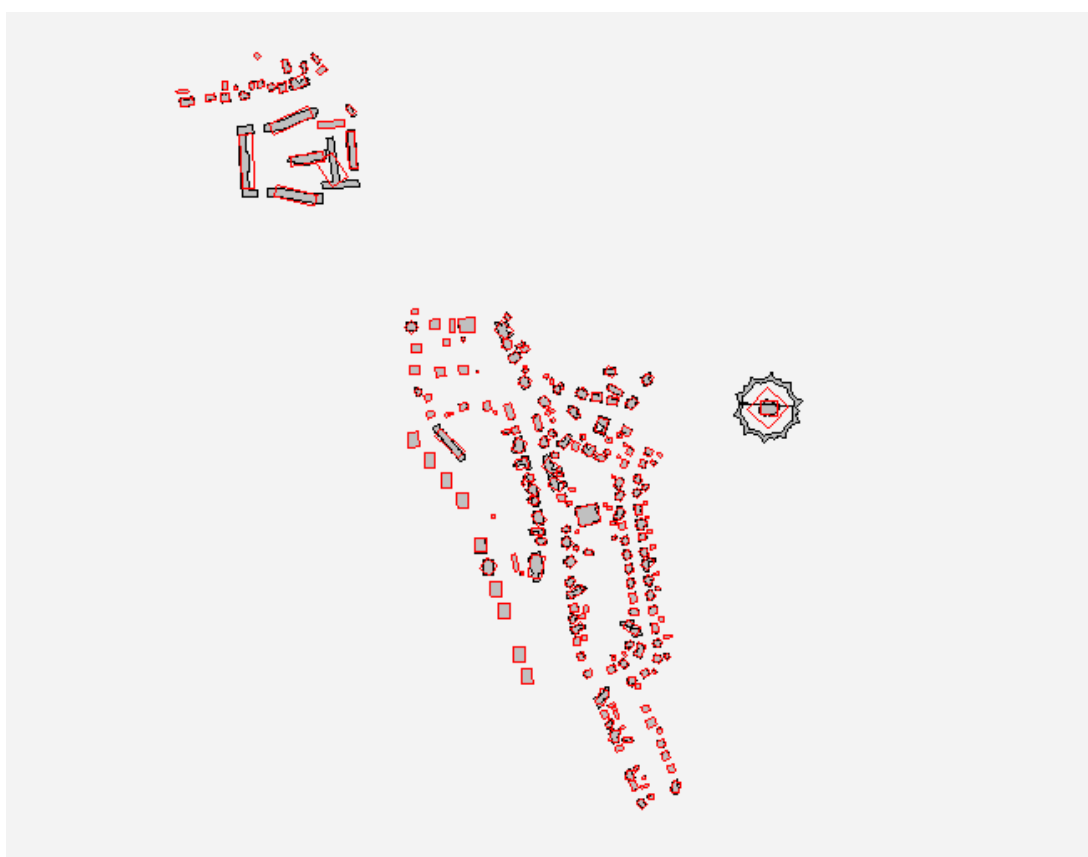
Obrázek 13: Printscreen výsledků generalizace metodou PCA nad daty z izolované zástavby.



Obrázek 14: Printscreen výsledků generalizace metodou Longest edge nad daty z izolované zástavby.



Obrázek 15: Printscreen výsledků generalizace metodou Wall Average nad daty z izolované zástavby.



Obrázek 16: Printscreen výsledků generalizace metodou Weighted Bisector nad daty z izolované zástavby.